

IA Investing OS v2

Projeto mestre de estabilização, refatoração e transformação em uma empresa de inteligência financeira

1. Escopo e conclusão da auditoria

Analisei estaticamente o branch público `main` do repositório. A auditoria abrangeu estrutura do código, configuração, banco de dados, conectores, agentes, workflows Temporal, métricas, scorecards, otimização, backtesting, RAG, API, infraestrutura, testes e modelo de produto. Não executei a stack completa de ponta a ponta; portanto, alguns bloqueadores de runtime ainda precisam ser confirmados por testes locais e de integração.

O README descreve uma plataforma com FastAPI, Temporal, OpenAI Agents SDK, PostgreSQL/pgvector, Polars, DuckDB, CVXPY, OpenTelemetry, MLflow e MinIO. A escolha tecnológica geral é boa, mas o código atual ainda é principalmente um **esqueleto técnico de prova de conceito**, não um sistema operacional de investimentos. Também não encontrei frontend no branch analisado. ([GitHub](#))

Minha recomendação não é descartar tudo. O caminho adequado é uma **refatoração evolutiva**, preservando as ideias úteis e substituindo gradualmente as partes frágeis.

O produto-alvo deve deixar de ser:

“Um conjunto de agentes que lê documentos e sugere ações.”

E passar a ser:

Um sistema operacional de investimentos orientado a evidências, que coleta dados, administra pesquisas, mantém teses versionadas, constrói carteiras-modelo, controla risco, executa comitês, monitora agentes e registra cada decisão de forma reproduzível e auditável.

2. Diagnóstico executivo

2.1 Nível de maturidade atual

As notas abaixo são estimativas de arquitetura baseadas no código público, não uma medição formal.

Área	Maturidade atual	Diagnóstico
Definição do produto	1/5	Existe visão técnica, mas não um modelo operacional completo
Backend/API	1,5/5	Rotas básicas, sem camada de aplicação, autenticação ou governança
Dados financeiros	1,5/5	Há conectores e modelos iniciais, mas falta confiança point-in-time
Workflows	1/5	Classes Temporal existem, porém a execução está incompleta
Agents de IA	1/5	Configurações e prompts iniciais, com problemas de importação e contratos
Pesquisa e teses	1,5/5	Bons conceitos iniciais, ainda não integrados em um ciclo operacional
Carteiras	1/5	CRUD e otimizador básico; falta mandato, NAV, snapshots e governança
Risco	0,5/5	Alguns campos e métricas, sem motor de risco operacional
Backtesting	1/5	Protótipo que ainda não deve sustentar decisões
Frontend	0/5	Não identificado no repositório
Segurança	0,5/5	Sem identidade, autorização, segregação de funções ou controle de ferramentas
Observabilidade	0,5/5	Estrutura inicial não integrada à aplicação
Testes	1/5	Alguns testes unitários, sem testes integrados ou de workflow
Compliance	0,5/5	Alguns conceitos de aprovação, mas sem perímetro regulatório implementado

2.2 Veredito

O sistema atual tem:

- Uma direção tecnológica apropriada.
- Alguns bons modelos conceituais.
- Um início de separação entre conectores, métricas, agentes e workflows.
- Código suficiente para servir como laboratório.

Porém, ainda não possui:

- Um backend executável e reproduzível de ponta a ponta.
- Contratos consistentes entre agentes, workflows e banco.

- Um modelo financeiro point-in-time confiável.
- Um verdadeiro domínio de carteiras.
- Um modelo de empresa financeira.
- Um painel operacional.
- Segurança e governança compatíveis com decisões de investimento.
- Um processo formal para transformar uma informação em decisão de carteira.

3. O que deve ser preservado, refatorado e removido

3.1 Preservar como direção

As seguintes escolhas são adequadas:

- Python para dados, IA e otimização.
- FastAPI como API.
- PostgreSQL como banco operacional.
- pgvector para recuperação semântica inicial.
- MinIO/S3 para arquivos brutos.
- Temporal para workflows duráveis.
- Pydantic para contratos de entrada e saída.
- Polars e DuckDB para processamento analítico.
- CVXPY para otimização.
- OpenTelemetry para observabilidade.
- O conceito de Raw Zone.
- O conceito de evidências, teses, avaliações e aprovações.
- O uso de agentes especializados.
- Métricas financeiras calculadas em código.

3.2 Refatorar profundamente

- Organização dos pacotes Python.
- Runtime dos agentes.
- Configuração de modelos.
- Banco de dados e migrations.
- Workflows Temporal.
- Conectores CVM, B3, RI, macro e notícias.
- Normalização de contas contábeis.
- Modelo de fatos financeiros.
- RAG.
- Modelo de carteira.
- Motor de risco.
- Otimizador.
- Backtesting.
- API.
- Infraestrutura local.
- Testes.
- Observabilidade.

3.3 Remover ou substituir

- Scheduler em loop de memória.
- `Base.metadata.create_all()` no startup.
- Rotas fazendo ORM diretamente.
- Execução síncrona de agentes em requisições HTTP.
- Dados de retorno enviados pelo cliente para o otimizador.
- Preço atual armazenado manualmente na posição.
- Modelos e preços de LLM hardcoded.
- Fallback silencioso do otimizador.
- Exceções amplas que convertem erro em resultado vazio.
- Auditoria "imutável" apenas por convenção.
- Campos essenciais armazenados exclusivamente em JSON.
- Agentes com acesso direto a escrita no banco.

- Qualquer possibilidade de ordem real antes da consolidação de paper trading, risco, aprovação e compliance.

4. Correções críticas identificadas

P0-01 — Colisão entre o pacote local `agents` e o OpenAI Agents SDK

O projeto possui um pacote local chamado `src/agents`. Dentro dele, `_runner.py` executa:

```
from agents import Agent, Runner
```

Como o próprio pacote local também se chama `agents`, existe risco direto de importação circular ou de o Python carregar o pacote local em vez do SDK externo. Esse é um provável bloqueador de inicialização. ([GitHub](#))

Correção

Renomear o pacote local:

```
src/agents
```

para:

```
src/ia_investing/ai
```

ou:

```
src/ia_investing/agent_runtime
```

Imports esperados:

```
from agents import Agent, Runner
from ia_investing.ai.registry import AgentRegistry
```

Critério de aceite

- Importar `Agent` e `Runner` em um teste isolado.
- Importar todos os módulos locais com `python -m compileall`.
- Executar um agente falso com modelo mockado.
- Executar `pytest --import-mode=importlib`.
- Nenhum pacote interno deve usar nomes que colidam com bibliotecas externas relevantes.

P0-02 — Caminhos de prompts incorretos e prompts ausentes

O runner define uma raiz apontando para `repo/prompts`, enquanto as configurações já usam caminhos iniciando em `prompts/...`. Isso tende a produzir um caminho equivalente a:

```
repo/prompts/prompts/...
```

Também há agentes configurados para os quais não existem diretórios ou prompts correspondentes, como coordenador, fundamentalista e diretor de risco. ([GitHub](#))

Correção

Não montar caminhos manualmente a partir de strings duplicadas.

```
PROMPTS_ROOT = Path(__file__).resolve().parents[3] / "prompts"
prompt_file = PROMPTS_ROOT / config.prompt_relative_path
```

O campo deve conter apenas:

```
filing_analyst/system.md
```

e não:

```
prompts/filing_analyst/system.md
```

Nova estrutura

```
prompts/
├─ research_coordinator/
│   └─ system.md
│   └─ output.schema.json
├─ filing_analyst/
├─ forensic_accounting/
├─ news_event_analyst/
├─ macro_analyst/
├─ political_regulatory_analyst/
├─ valuation_analyst/
├─ risk_director/
├─ portfolio_architect/
├─ critic/
├─ committee_chair/
└─ compliance_reviewer/
```

Critério de aceite

- Teste que percorre todo o registro de agentes.
- Cada agente deve possuir prompt, schema, versão e testes.
- Inicialização deve falhar imediatamente se um arquivo estiver ausente.
- Prompt e schema devem ser identificados por hash e versão.

P0-03 — Configuração inconsistente e dependências ausentes

O README orienta a criar `.env` na raiz. A configuração procura o arquivo em `src/.env`. O banco usa URL `postgresql+asyncpg`, mas `asyncpg` não aparece como dependência direta no projeto analisado. Configurações como OpenAI base URL, LiteLLM e políticas de modelo também não são efetivamente aplicadas pelo runner. (GitHub)

Correção

Criar uma única convenção:

```
model_config = SettingsConfigDict(
    env_file=".env",
    env_nested_delimiter="__",
    case_sensitive=False,
)
```

Organizar configurações:

```
DatabaseSettings
StorageSettings
TemporalSettings
AISettings
TelemetrySettings
SecuritySettings
ApplicationSettings
```

Regras

- Dependências diretas devem ser declaradas diretamente.
- Criar `uv.lock`.
- Separar `.env.example`, `.env.test` e variáveis de produção.
- Não usar valores default inseguros em produção.
- Falhar na inicialização quando credenciais obrigatórias não estiverem presentes.
- Nunca apenas emitir `warnings.warn()` para configurações críticas.

Critério de aceite

```
uv sync --frozen
python -m ia_investing.cli check-config
```

deve funcionar em ambiente limpo.

P0-04 — Migrations inexistentes e conflito entre Alembic e `create_all`

O startup da API chama `Base.metadata.create_all()`, enquanto o README orienta executar Alembic. O repositório possui o ambiente do Alembic, mas não há um conjunto de revisões capaz de construir a estrutura atual. A URL também está hardcoded no `alembic.ini`, sem seguir adequadamente a configuração central. (GitHub)

Correção

- Remover `create_all()` da API.
- Fazer a aplicação verificar a versão da migration.
- Criar uma migration inicial consolidada.
- Aplicar migrations em um job separado de deploy.
- Usar a mesma `Settings.database_url`.
- Adicionar convenção de nomes para índices e constraints.

Exemplo:

```
NAMING_CONVENTION = {
    "ix": "ix_%(table_name)s_%(column_0_name)s",
    "uq": "uq_%(table_name)s_%(column_0_name)s",
    "ck": "ck_%(table_name)s_%(constraint_name)s",
    "fk": "fk_%(table_name)s_%(column_0_name)s_%(referred_table_name)s",
    "pk": "pk_%(table_name)s",
}
```

Critério de aceite

Os seguintes fluxos devem funcionar:

```
alembic upgrade head
alembic downgrade -1
alembic upgrade head
alembic check
```

Um banco vazio precisa ser construído somente por migrations.

P0-05 — Campos SQLAlchemy aparentam não estar mapeados

Há diversos campos declarados como:

```
line_items = JSONB()
raw_data = JSONB()
parsed_data = JSONB()
payload = JSONB()
canonical_data = JSONB()
```

em vez de `Column(JSONB)` ou `mapped_column(JSONB)`. Na forma atual, esses atributos não seguem o padrão normal de mapeamento declarativo e podem não ser criados ou persistidos como esperado. ([GitHub](#))

Correção

Migrar os modelos para SQLAlchemy 2 tipado:

```
class FinancialStatement(Base):
    line_items: Mapped[dict[str, Any]] = mapped_column(
        JSONB,
        nullable=False,
        default=dict,
    )
```

Adicionalmente

- Tornar campos de negócio obrigatórios `nullable=False`.
- Criar `CheckConstraint` para scores entre zero e um.
- Usar enums controlados para status.
- Criar índices parciais.
- Definir constraints de unicidade.
- Evitar usar JSONB para fatos financeiros centrais.

Critério de aceite

- Alembic deve detectar todos os campos.
- Teste de round-trip deve inserir, carregar e comparar o JSON.
- Testes devem consultar chaves e índices JSONB quando aplicável.

P0-06 — Worker Temporal não registra activities

O worker registra workflows, mas não registra as activities usadas pelos workflows. No Temporal, o worker precisa registrar explicitamente os tipos exatos de workflows e activities associados à task queue. ([GitHub](#))

Correção

Separar workers por capacidade:

```
data-ingestion
document-processing
research-agents
portfolio-risk
notifications
```

Exemplo:

```
worker = Worker(
    client,
    task_queue="data-ingestion",
    workflows=[FilingPublishedWorkflow],
    activities=[
        discover_filing,
        download_source_object,
        persist_raw_object,
        parse_cvm_dataset,
        validate_financial_facts,
        publish_outbox_event,
    ],
)
```

Políticas obrigatórias

- Retry por activity.
- Erros não retentáveis para schema inválido.
- Heartbeat em downloads e parsers longos.
- Idempotency key.
- Timeout de início e execução.
- Dead-letter ou quarentena de negócio.
- Métricas por activity.

Activities podem ser executadas mais de uma vez, ainda que o workflow observe uma conclusão. Por isso, escritas e chamadas externas devem ser idempotentes. ([docs.temporal.io](#))

Critério de aceite

- Workflows executados no ambiente de teste Temporal.
- Replay test dos históricos.
- Activity repetida não deve duplicar documento, métrica ou evento.
- Falha de worker durante uma activity deve permitir retomada segura.

P0-07 — O scheduler atual não agenda de fato os workflows

O scheduler mantém loops em memória. Em alguns casos apenas instancia classes de workflow ou escreve logs. O processo macro coleta dados, mas aparentemente não os persiste. O estado de agendamento também é perdido ao reiniciar. ([GitHub](#))

Correção

Eliminar o scheduler customizado e usar **Temporal Schedules**.

Temporal Schedules oferece identidade própria, intervalos e calendários, políticas de sobreposição, catch-up, pausa, backfill e pause-on-failure. ([docs.temporal.io](#))

Schedules iniciais

source.cvm.discovery	a cada 20 minutos
source.b3.end_of_day	após fechamento
source.bcb.macro	diariamente
source.news.monitoring	a cada 15 minutos
source.policy.monitoring	a cada 30 minutos
portfolio.daily_valuation	após consolidação do mercado
risk.intraday	a cada hora durante mercado
opportunity.weekly_screen	semanal
committee.weekly_pack	semanal

agent.evaluation	após mudança de modelo/prompt
data.quality.reconciliation	diariamente

Critério de aceite

- Schedules visíveis no Temporal UI.
- Política de overlap configurada.
- Reinicialização não perde agenda.
- Backfill pode reconstruir uma janela histórica.
- Falhas críticas pausam o schedule e geram alerta.

P0-08 — Contratos incompatíveis entre schemas, prompts e workflows

O schema de notícia retorna campos como `verdict`, `summary_pt`, `thesis_effect` e `key_claims`. O workflow procura `description`, `direction_hint`, `affected_issuers` e `agent_run_id`. Consequentemente, um resultado válido do agente pode ser convertido em `unknown`, listas vazias ou valores default. (GitHub)

O schema de comitê possui somente decisão, confiança, texto, condições e dissenso. Ele não representa ação, tamanho da posição, carteira, cenário, risco, preço, prazo ou critérios de revisão. (GitHub)

Correção

Criar contratos canônicos compartilhados entre:

- API.
- Agent runtime.
- Workflow.
- Banco.
- Frontend.
- Evals.

Nunca manter uma dataclass local duplicando um schema Pydantic.

Contrato mínimo de uma análise

```
{
  "analysis_id": "uuid",
  "case_id": "uuid",
  "agent_version_id": "uuid",
  "data_as_of": "2026-07-18T10:00:00-03:00",
  "verdict": "positive",
  "confidence": {
    "model": 0.81,
    "evidence_coverage": 0.92,
    "data_quality": 0.88
  },
  "claims": [],
  "facts": [],
  "inferences": [],
  "assumptions": [],
  "risks": [],
  "affected_metrics": [],
  "evidence_ids": [],
  "contradictions": [],
  "recommendation": null,
  "expires_at": "2026-08-18T00:00:00-03:00"
}
```

Critério de aceite

- Um único schema por mensagem de domínio.
- Testes de serialização entre activity, workflow, banco e API.
- OpenAPI gerado e validado.
- Nenhum `.get("campo", default)` em contratos obrigatórios.
- Mudança incompatível exige nova versão de schema.

P0-09 — API sem segurança e com operações longas no request

A API não possui autenticação, autorização, identificação de organização, segregação de função, rate limit ou contexto de auditoria. As rotas acessam diretamente o ORM e a rota de agentes executa o LLM dentro da requisição HTTP. (GitHub)

Correção

Separar:

```
HTTP route
  -> command/query handler
    -> application service
      -> domain
        -> repository/infrastructure
```

Uma execução de agente deve retornar:

```
202 Accepted
Location: /v1/operations/{operation_id}
```

E iniciar um workflow Temporal.

Critério de aceite

- OIDC/OAuth2 obrigatório.
- Cada endpoint possui permissão explícita.
- Cada comando gera `audit_event`.
- Operações longas usam `202`.
- Nenhuma rota chama diretamente um LLM.
- Nenhuma rota realiza otimização pesada no event loop da API.

P0-10 — Erro no filtro de setor da API

A rota de emissores usa `Issuer.industry`, mas o modelo analisado não define esse relacionamento. O filtro por setor tende a falhar em runtime. ([GitHub](#))

Correção

Adicionar relacionamento tipado ou escrever o join explicitamente:

```
stmt = (
  select(Issuer)
    .join(Industry, Issuer.industry_id == Industry.id)
    .join(Sector, Industry.sector_id == Sector.id)
)
```

Critério de aceite

- Teste integrado da consulta.
- Paginação com total.
- Filtros combináveis.
- Busca por ticker, CNPJ, nome, setor e alias.
- `EXPLAIN ANALYZE` dentro do limite esperado.

P0-11 — Integridade dos dados CVM insuficiente

O conector financeiro cobre apenas parte dos demonstrativos e o tratamento de parsing pode converter falha ou ausência em zero. Em finanças, "não encontrado", "não aplicável" e "zero" são estados diferentes. O conector também tende a baixar e processar arquivos anuais inteiros repetidamente. ([GitHub](#))

A normalização atual depende de um mapa estático genérico. A própria CVM disponibiliza dicionários e metadados oficiais para os conjuntos DFP, que devem ser tratados como contrato de fonte versionado. ([Portal Dados Abertos CVM](#))

Correção

Representar valor como:

```
value
value_status:
  reported
  calculated
  missing
  not_applicable
  parse_error
  suppressed
```

Cobrir:

- BPA.

- BPP.
- DRE.
- DFC método direto.
- DFC método indireto.
- DMPL.
- DVA.
- Informações por segmento.
- Dados consolidados e individuais.
- Reapresentações.
- Versões de formulário.
- Moeda e escala.
- Contas criadas pelo emissor.

Critério de aceite

- Nenhum parse error vira zero.
- Reapresentação cria nova versão.
- Dados individuais e consolidados não se misturam.
- Dicionário de dados da fonte possui versão registrada.
- Amostra validada contra documentos oficiais de diferentes setores.
- Reconciliações contábeis são executadas antes da promoção para dados canônicos.

P0-12 — Modelo financeiro excessivamente baseado em JSONB

As demonstrações normalizadas são armazenadas principalmente como um documento JSON. Isso pode ser útil como payload bruto, mas é insuficiente como fonte canônica de fatos financeiros, porque prejudica:

- Constraints.
- Linhagem por conta.
- Comparação entre versões.
- Restatements.
- Agregações.
- Auditoria de fórmulas.
- Consultas point-in-time.

([GitHub](#))

Correção

Criar uma tabela de fatos:

```
financial_fact
- id
- issuer_id
- instrument_scope
- reporting_period_id
- statement_type
- consolidation_scope
- account_taxonomy_id
- source_account_code
- source_account_label
- value
- currency
- scale
- value_status
- published_at
- knowledge_at
- source_object_version_id
- parser_version
- mapping_rule_version_id
- valid_from
- valid_to
- revision_number
```

O JSON original continua preservado na Raw Zone, mas as análises usam fatos canônicos.

P0-13 — Modelo de carteira não representa uma carteira real

O modelo atual de carteira contém nome, descrição, flag de paper trading, moeda e capital inicial. A posição mantém ticker em texto, quantidade, custo e preço atual mutável. Faltam:

- Mandato.
- Estratégia.
- Benchmark.
- Horizonte.
- Política de risco.
- Caixa.
- NAV.
- Snapshots.
- Versões.
- Lotes.
- Proventos.
- Corporate actions.
- Custos.
- Impostos.
- Ordens.
- Fills.
- Reconciliação.
- Responsáveis.
- Aprovações.
- Estado operacional.

([GitHub](#))

Correção

Separar:

1. Mandato de investimento
2. Carteira-modelo
3. Versão aprovada
4. Carteira paper
5. Carteira live
6. Snapshots de posição
7. NAV e performance
8. Propostas de rebalanceamento
9. Intenções de negociação
10. Ordens e execuções

Nunca armazenar “preço atual” como propriedade permanente da posição. O valor da posição deve ser calculado a partir de:

```
position_snapshot.quantity  
x  
market_price válido no as_of
```

P0-14 — Otimizador pode retornar uma solução aparentemente válida, mas inadequada

O otimizador é chamado dentro de uma função assíncrona, embora a solução CVXPY seja CPU-bound. O fallback equal-weight pode violar as próprias restrições. Além disso:

- O cliente envia a matriz de retornos.
- Não existe validação de qualidade dos dados.
- O peso máximo padrão de 10% torna o problema inviável para universos com menos de dez ativos.
- Não há caixa, liquidez, lote, custos, impostos ou capacidade.
- Não há diagnóstico detalhado de infeasibility.

([GitHub](#))

Correção

- Executar otimização em worker dedicado.
- Construir dados exclusivamente no backend.
- Validar viabilidade antes de chamar solver.
- Usar covariance shrinkage ou modelo fatorial.
- Persistir versão dos inputs.
- Persistir solver, status, tolerâncias e resultado.
- Falhar de forma fechada.

Nunca fazer:

```
solver falhou -> retornar equal-weight como se fosse solução
```

Deve retornar:

```
{
  "status": "infeasible",
  "violated_constraints": [],
  "diagnostics": [],
  "relaxation_suggestions": []
}
```

P0-15 — Backtest com riscos de erro lógico e look-ahead

No código atual, a coluna do benchmark pode entrar no conjunto de ativos investíveis. O índice de cálculo de retornos também aparenta um deslocamento que omite ou atrasa observações. Além disso, o backtest não contém o conjunto mínimo necessário para servir de prova econômica: universo histórico, datas reais de publicação, deslistagens, proventos, custos, impostos, liquidez e corporate actions. ([GitHub](#))

Correção mínima

- Benchmark fora do universo investível.
- Datas de publicação, não somente datas do período contábil.
- Universo reconstruído em cada data.
- Preços ajustados por evento, sem introduzir informação futura.
- Delistings e empresas extintas.
- Custo, spread, slippage e impostos configuráveis.
- Delay entre sinal e execução.
- Walk-forward.
- Período out-of-sample.
- Comparação com baselines.
- Testes anti-look-ahead.

Gate

Nenhuma carteira pode receber o status `eligible_for_paper` enquanto o backtest não passar em testes de point-in-time e reprodutibilidade.

P0-16 — RAG não preserva evidência suficiente

O chunker trabalha por quantidade de caracteres, sem respeitar página, seção ou tabela. A metadata criada pelo RAG não é persistida no modelo de embedding. O modelo tem dimensão fixa, sem versionamento do modelo de embeddings. A busca também não possui filtro temporal, limiar, busca híbrida ou reranking. ([GitHub](#))

O parser de PDF concatena o texto de todas as páginas, o que impede uma citação confiável por página e região. ([GitHub](#))

Correção

Cada chunk deve possuir:

```
document_version_id
page_start
page_end
section_path
chunk_ordinal
content_hash
text
table_reference
embedding_model
embedding_dimensions
embedding_version
created_at
```

Pipeline

```
documento
-> extração por página
-> detecção de layout
-> detecção de seções
-> tabelas separadas
-> chunk semântico
-> embedding
-> busca híbrida
```

```
-> reranking
-> evidência citável
```

Regra

Nenhum claim importante pode ser considerado “verificado” sem `evidence_id`.

P0-17 — Observabilidade não está conectada ao sistema

Há uma configuração inicial de OpenTelemetry, mas a API não parece inicializá-la. O collector exporta basicamente para logging, sem backend persistente de traces, métricas ou logs. ([GitHub](#))

Correção

Implementar:

- OpenTelemetry SDK na API, workers e connectors.
- Instrumentação de FastAPI.
- SQLAlchemy.
- HTTPX.
- Temporal.
- Chamadas de modelo.
- Eventos de domínio.
- Prometheus.
- Grafana.
- Tempo.
- Loki.
- Alertmanager.
- Sentry opcional para exceções.

Correlação obrigatória

```
source_object_id
workflow_id
activity_id
agent_run_id
research_case_id
thesis_version_id
committee_decision_id
portfolio_version_id
rebalance_proposal_id
order_id
```

P0-18 — Infraestrutura local incompleta

O `docker-compose.yml` contém infraestrutura, mas não inclui de forma completa API, workers, scheduler e frontend. Há imagens com tag `latest`, o que reduz reprodutibilidade. A configuração do MLflow também depende de um banco que não aparenta ser criado pelo compose atual. ([GitHub](#))

Correção

Compose local:

```
postgres
minio
minio-init
temporal
temporal-ui
api
worker-data
worker-research
worker-portfolio
web
otel-collector
prometheus
grafana
tempo
loki
mlflow
migration
```

Regras

- Fixar versões de imagens.
- Criar buckets automaticamente.
- Criar bancos automaticamente.
- Healthchecks.
- Dependências condicionadas a health.
- Volumes nomeados.
- Perfil `dev`, `test` e `observability`.
- Usuários de banco distintos por serviço.
- Container sem root.
- Read-only filesystem quando possível.

P0-19 — Cobertura de testes insuficiente

O repositório possui um conjunto pequeno de testes unitários para parser, métricas, normalização, scorecard, schemas e otimizador. Não encontrei testes de API, banco, migrations, Temporal, RAG, segurança, contratos de fonte, frontend ou execução integrada. ([GitHub](#))

Nova pirâmide de testes

```
tests/
├─ unit/
├─ property/
├─ integration/
├─ contract/
├─ workflow/
├─ replay/
├─ api/
├─ security/
├─ backtest/
├─ evals/
├─ golden_documents/
└─ e2e/
```

Gate

Não aceitar mudanças em:

- Fórmulas financeiras sem golden tests.
- Conectores sem contract tests.
- Workflows sem replay tests.
- Prompts sem evals.
- Schemas sem compatibility test.
- Frontend sem testes de acessibilidade e E2E.

P0-20 — Questão de licenciamento de conteúdo no repositório

O repositório público contém um PDF de livro em `docs/books`. É necessário validar se existe autorização para armazenar e redistribuir esse conteúdo. Caso não haja licença explícita, o arquivo deve ser removido do histórico público e substituído por referência bibliográfica ou anotações próprias. ([GitHub](#))

5. Projeto-alvo: IA Investing OS

5.1 Objetivo

Construir uma plataforma que opere como uma **empresa de inteligência e gestão de investimentos assistida por IA**, com as seguintes capacidades:

1. Monitorar mercado, empresas, política, macroeconomia e regulação.
2. Identificar eventos e oportunidades.
3. Criar casos de pesquisa.
4. Produzir análises com evidências.
5. Manter teses versionadas.
6. Estimar cenários e valuation.
7. Analisar riscos.
8. Construir carteiras-modelo.
9. Realizar comitês.
10. Aprovar ou rejeitar propostas.
11. Operar inicialmente em paper trading.

12. Medir a qualidade das decisões.
13. Revisar teses e carteiras continuamente.
14. Explicar por que cada ativo está ou não está na carteira.

5.2 Não objetivos da primeira versão

- Execução autônoma de ordens reais.
- Recomendação individualizada para clientes.
- Gestão discricionária de recursos de terceiros.
- Dados de baixa latência ou negociação de alta frequência.
- Dezenas de microserviços.
- Kafka antes de existir demanda real.
- Kubernetes no ambiente inicial.
- Fine-tuning antes de possuir datasets confiáveis.
- Um “agente CEO” com liberdade irrestrita.
- Um ranking global de carteiras baseado somente em rentabilidade.

6. Princípios obrigatórios do produto

6.1 Evidência antes de opinião

Toda conclusão material precisa indicar:

- Fato observado.
- Fonte.
- Versão da fonte.
- Momento em que o dado ficou disponível.
- Interpretação.
- Assumptions.
- Confiança.
- Agente ou usuário responsável.

6.2 Separar fato, inferência e recomendação

```
Fato:  
A companhia publicou redução de guidance.  
  
Inferência:  
A redução pode comprometer a expansão de margem prevista na tese.  
  
Recomendação:  
Revisar o cenário-base e suspender novas compras até o novo valuation.
```

Esses três elementos não podem ser armazenados como um único parágrafo indistinto.

6.3 Matemática determinística; IA para ambiguidade

Código calcula:

- Margens.
- Crescimento.
- ROIC.
- Múltiplos.
- Volatilidade.
- Exposições.
- Risco.
- NAV.
- Performance.
- Otimização.
- Backtest.

Agents analisam:

- Mudança de narrativa.
- Qualidade da administração.
- Risco regulatório.

- Contradições.
- Materialidade de notícias.
- Relações entre eventos.
- Cenários qualitativos.
- Força ou enfraquecimento da tese.

6.4 Point-in-time em todos os níveis

O sistema precisa saber:

- A qual período o dado se refere.
- Quando foi publicado.
- Quando foi descoberto.
- Quando foi ingerido.
- Quando foi validado.
- Qual versão estava vigente.
- Qual versão o processo de decisão conhecia.

6.5 Agents não controlam o sistema

- Temporal controla o workflow.
- Serviços controlam dados e cálculos.
- Políticas controlam permissões.
- Agents produzem avaliações estruturadas.
- Humanos aprovam efeitos sensíveis.

6.6 Toda decisão expira

Uma recomendação precisa possuir:

- `valid_from`.
- `expires_at`.
- Data de revisão.
- Eventos de invalidação.
- Condições pendentes.

6.7 Falhar fechado

Quando dados, solver, modelo, fonte ou evidência falham:

```
status = unavailable | incomplete | blocked
```

Nunca:

```
falhou -> gerar uma recomendação genérica
```

7. Modelo operacional da “empresa financeira”

7.1 Escritório de dados

Responsabilidades

- Cadastro de emissores e instrumentos.
- Coleta.
- Raw Zone.
- Normalização.
- Reconciliação.
- Qualidade.
- Linhagem.
- Reapresentações.
- Licenciamento de fontes.

Automação

- Conectores.
- Parsers.
- Data quality rules.
- Entity resolution.
- Source registry.
- Quarentena.

Agent associado

Data Steward Agent

Não calcula decisão de investimento. Ele:

- Classifica erros.
- Sugere mapeamentos de contas.
- Detecta anomalias semânticas.
- Compara labels novos com taxonomia.
- Encaminha casos ambíguos para revisão humana.

7.2 Pesquisa de empresas

Responsabilidades

- Análise de DFP, ITR, FRE, fatos relevantes e releases.
- Qualidade dos resultados.
- Modelo de negócio.
- Governança.
- Concorrência.
- Drivers e riscos.
- Tese.

Agents

- Filing Analyst.
- Forensic Accounting Analyst.
- Sector Analyst.
- Governance Analyst.
- Valuation Analyst.
- Devil's Advocate.

Saída

Um `ResearchCase` que pode gerar ou atualizar uma `InvestmentThesisVersion`.

7.3 Inteligência de notícias e eventos

Responsabilidades

- Descobrir notícias.
- Deduplicar.
- Identificar entidades.
- Classificar evento.
- Corroborar fontes.
- Medir novidade e materialidade.
- Comparar com teses e posições.

Agents

- News Event Analyst.
- Event Corroboration Agent.
- Thesis Impact Analyst.

Saída

Evento **verificado**
Evento **não confirmado**
Evento **contraditório**

Rumor
Atualização sem materialidade

Sentimento não deve ser o elemento principal. O sistema deve perguntar:

- O que aconteceu?
- A informação é nova?
- A fonte é confiável?
- O evento foi confirmado?
- Qual métrica pode mudar?
- Qual tese é afetada?
- Qual horizonte importa?

7.4 Inteligência macroeconômica

Responsabilidades

- Juros.
- Inflação.
- Câmbio.
- Crédito.
- Fiscal.
- Emprego.
- Commodities.
- Curva de juros.
- Regimes macro.
- Sensibilidades setoriais.

Agents

- Macro Economist.
- Regime Classification Agent.
- Cross-Asset Analyst.

Saída

- Regime atual.
- Cenários.
- Probabilidades.
- Variáveis críticas.
- Setores beneficiados e prejudicados.
- Impacto potencial nas carteiras.

7.5 Inteligência política e regulatória

Esse domínio não deve ser tratado como "análise de sentimento político".

Fontes iniciais

A Câmara disponibiliza dados de proposições, temas, eventos, votações, parlamentares e órgãos. O Senado disponibiliza dados de parlamentares, comissões, sessões, processos legislativos, votações, discursos e orçamento; sua API também informa limites e condições de requisição que devem ser respeitados.

(dadosabertos.camara.leg.br)

O INLABS disponibiliza publicações do Diário Oficial da União em XML e PDF, ressaltando que o XML não substitui a versão certificada. (inlabs.in.gov.br)

Outros conectores necessários

- Planalto e legislação.
- Banco Central.
- CMN e Copom.
- Ministério da Fazenda.
- Tesouro Nacional.
- Receita Federal.
- CADE.
- ANEEL.
- ANP.

- ANATEL.
- ANS.
- SUSEP.
- CVM.
- TCU.
- IBAMA.
- Agências e ministérios específicos por setor.

Modelo de evento político

```
policy_proposal
- tipo
- número
- autor
- texto
- tema
- estágio
- comissão
- relator
- próximos prazos
- probabilidade de avanço
- probabilidade de aprovação
- vigência esperada
- setores afetados
- empresas expostas
- cenários
- evidências
```

Agent

Political & Regulatory Analyst

Ele deve:

1. Resumir o conteúdo.
2. Identificar estágio real.
3. Diferenciar proposta, aprovação e vigência.
4. Mapear stakeholders.
5. Estimar cenários.
6. Identificar setores e companhias expostas.
7. Comparar com teses.
8. Explicitar incerteza.
9. Nunca apresentar preferência partidária como análise financeira.

7.6 Quant e valuation

Responsabilidades

- Fatores.
- Scorecards.
- Comparação de pares.
- DCF.
- Reverse DCF.
- Cenários.
- Retorno esperado.
- Pesquisa quantitativa.
- Robustez.

Agents

- Quant Research Analyst.
- Valuation Analyst.

Serviços determinísticos

- Metric Engine.
- Factor Engine.
- Peer Ranking.
- Scenario Engine.
- DCF Engine.

- Backtest Engine.
-

7.7 Risco

Responsabilidades

- Limites.
- Concentração.
- Liquidez.
- Fatores.
- Correlação.
- VaR e CVaR.
- Stress.
- Drawdown.
- Risco político.
- Risco regulatório.
- Risco de dados.
- Risco de modelo.

Agent

Risk Director Agent

O agent explica e prioriza riscos. O motor quantitativo calcula os números.

Autoridade

O domínio de risco pode bloquear:

- Entrada de ativo.
 - Aumento de posição.
 - Rebalanceamento.
 - Ativação live.
 - Uso de dados inconsistentes.
-

7.8 Construção de carteiras

Responsabilidades

- Selecionar universo.
- Definir retorno esperado por cenário.
- Aplicar restrições.
- Otimizar.
- Simular custos.
- Comparar alternativas.
- Produzir proposta.

Agent

Portfolio Architect

Não inventa pesos. Ele:

- Escolhe cenários aprovados.
 - Solicita execução do otimizador.
 - Interpreta trade-offs.
 - Compara propostas.
 - Produz justificativa.
-

7.9 Comitê de investimentos

Responsabilidades

- Avaliar tese.
- Avaliar contra-tese.
- Revisar risco.

- Revisar valuation.
- Revisar política.
- Votar.
- Condicionar.
- Aprovar ou rejeitar.

Agents

- Committee Chair.
- Devil’s Advocate.
- Compliance Reviewer.
- Risk Director.
- Portfolio Architect.

Humano

Pelo menos uma aprovação humana é obrigatória em paper trading material. Para operação real, deve ser aplicada segregação de funções e regra de quatro olhos.

7.10 Operações

Responsabilidades futuras

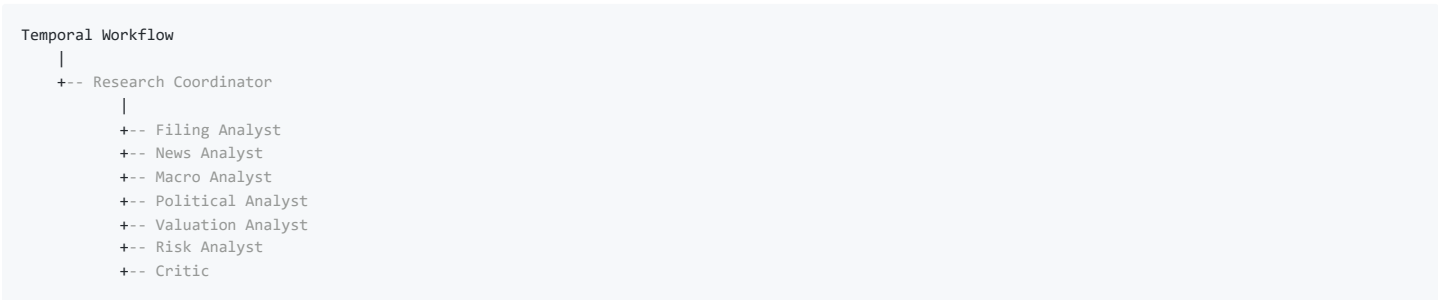
- Converter proposta aprovada em intenção de negociação.
- Gerar ordens.
- Receber fills.
- Reconciliar.
- Monitorar divergência.
- Cancelar.
- Aplicar kill switch.

Agents nunca devem acessar diretamente a credencial da corretora.

8. Framework de agents

8.1 Arquitetura recomendada

Use um coordenador controlado por workflow e especialistas como ferramentas para tarefas limitadas. No Agents SDK, o padrão “agents as tools” mantém um agente gerente no controle, enquanto handoffs transferem a condução para o especialista. Para este sistema, especialistas como ferramentas são mais apropriados na maior parte dos processos de backend. (OpenAI GitHub)



8.2 Catálogo lógico de agents

Agent	Entrada	Ferramentas	Saída	Gate
Research Coordinator	Caso e contexto	Busca de evidências e especialistas	Plano de pesquisa	Policy engine
Data Steward	Registros problemáticos	Taxonomia e histórico	Sugestão de mapeamento	Revisão de dados
Filing Analyst	Documento e métricas	RAG, fatos financeiros	Mudanças e claims	Evidence validator
Forensic Accounting	Demonstrativos	Regras contábeis	Alertas e inconsistências	Data quality
Sector Analyst	Empresa e pares	Métricas setoriais	Posicionamento competitivo	Research lead
News Event Analyst	Conteúdo e fonte	Entity resolver	Evento estruturado	Corroboração
Macro Economist	Séries e eventos	Scenario engine	Regime e cenários	Macro review
Political Analyst	Propostas e atos	Legislative graph	Impacto regulatório	Source corroboration

Agent	Entrada	Ferramentas	Saída	Gate
Valuation Analyst	Métricas e cenário	DCF e pares	Faixa de valor	Formula validation
Quant Analyst	Universo e fatores	Factor engine	Ranking e robustez	Backtest gate
Risk Director	Carteira e cenários	Risk engine	Parecer de risco	Hard limits
Devil’s Advocate	Tese completa	Evidências opostas	Contra-tese	Obrigatório
Portfolio Architect	Candidatos	Optimizer	Proposta de alocação	Risk gate
Committee Chair	Pack completo	Votos e políticas	Decisão	Humano
Compliance Reviewer	Ação proposta	Policies e restricted list	Liberação ou bloqueio	Obrigatório
Post-Mortem Analyst	Resultado realizado	Histórico da decisão	Erros e aprendizados	Model governance

Esses agents são papéis lógicos. Não precisam ser microserviços separados.

8.3 Contrato de execução

Toda execução deve registrar:

```
run_id
workflow_id
case_id
agent_definition_id
agent_version
prompt_version
model_profile
schema_version
data_as_of
knowledge_cutoff
input_hash
input_evidence_ids
output_hash
claims
citations
assumptions
uncertainties
contradictions
tokens
cost
latency
status
failure_reason
human_review
```

8.4 Ferramentas tipadas

Agents não devem receber:

- SQL arbitrário.
- Shell.
- Credenciais.
- Escrita irrestrita.
- Navegação irrestrita.
- Corretora.

Devem receber ferramentas como:

```
get_financial_metrics(
    issuer_id: UUID,
    as_of: datetime,
    metric_names: list[str],
) -> MetricBundle

search_evidence(
    issuer_id: UUID,
    query: str,
    as_of: datetime,
) -> list[EvidenceReference]

calculate_valuation(
    issuer_id: UUID,
    scenario_id: UUID,
) -> ValuationResult

request_thesis_update(
    thesis_id: UUID,
```

```
change_set: ThesisChangeSet,  
) -> CommandReceipt
```

8.5 Guardrails

O SDK suporta guardrails de entrada, saída e ferramentas. Tool guardrails devem ser usados em cada chamada sensível, e não somente no primeiro e último agente do fluxo. ([OpenAI GitHub](#))

Guardrails necessários:

- Validação de schema.
- Verificação de citation IDs.
- Separação de fato e inferência.
- Verificação de datas.
- Limite de materialidade.
- Restricted list.
- Prompt injection.
- Dado pessoal.
- Orçamento de tokens.
- Máximo de turnos.
- Domínios permitidos.
- Proibição de ordem.
- Proibição de alterar fatos canônicos.

8.6 Aprovação humana

Chamadas sensíveis devem poder pausar e aguardar aprovação. O Agents SDK suporta interrupção, serialização e retomada de runs com aprovação ou rejeição de ferramentas. ([OpenAI GitHub](#))

Exemplos:

update_active_thesis	requer aprovação do analista
submit_rebalance_proposal	requer aprovação do gestor
approve_model_portfolio	requer comitê
create_trade_intent	requer risco + operações
send_order	requer regras específicas e não entra no MVP

8.7 Tracing

O Agents SDK registra gerações, tool calls, handoffs, guardrails e eventos customizados. Esse tracing deve ser correlacionado com os traces OpenTelemetry e com os IDs de domínio. ([OpenAI GitHub](#))

9. Framework das “Top X carteiras”

9.1 Não criar um ranking global ingênuo

Uma carteira de dividendos não deve competir diretamente com uma carteira small cap de crescimento ou com uma carteira defensiva.

O painel precisa mostrar:

```
Top 5 – Renda  
Top 5 – Qualidade  
Top 5 – Value  
Top 5 – Crescimento  
Top 5 – Baixa volatilidade  
Top 5 – Small caps  
Top 5 – Macro  
Top 5 – Event-driven  
Top 5 – Customizadas
```

Cada ranking deve possuir:

- Mesmo benchmark ou benchmarks comparáveis.
- Mesmo horizonte.
- Mesmo nível de risco.
- Mesma moeda.
- Mesmo estágio operacional.
- Mesmas regras de elegibilidade.

9.2 Tipos de carteira

Carteira-modelo

Representa uma estratégia e uma alocação teórica aprovada.

Carteira paper

Simula ordens, custos, proventos e execução.

Carteira live

Espelha uma conta real ou recebe dados de uma corretora.

Essas três carteiras nunca devem compartilhar a mesma tabela mutável de posições.

9.3 Mandato obrigatório

Cada carteira precisa de:

```
name
objective
strategy_type
investment_universe
benchmark
base_currency
investment_horizon
rebalance_frequency
risk_budget
target_volatility
max_drawdown_tolerance
max_position
max_sector
max_factor_exposure
min_liquidity
cash_range
turnover_limit
excluded_assets
tax_profile
cost_assumptions
approval_policy
```

9.4 Estados

```
draft
-> researching
-> simulated
-> committee_review
-> approved
-> paper_live
-> eligible_for_live
-> live
-> suspended
-> archived
```

Não permitir saltos sem transição registrada.

9.5 Elegibilidade para o ranking

Uma carteira só pode participar da seção Top X quando:

- Possuir mandato válido.
- Possuir histórico mínimo configurado.
- Estar em paper ou live.
- Não possuir violação crítica aberta.
- Possuir NAV reconciliado.
- Possuir benchmark completo.
- Ter dados e métricas atualizados.
- Passar por backtest point-in-time.
- Possuir versão aprovada pelo comitê.
- Não estar suspensa.
- Possuir evidência suficiente para as maiores posições.

9.6 Score sugerido

Exemplo inicial, configurável por mandato:

Componente	Peso
Retorno excedente líquido	20%
Sortino e eficiência de risco	15%
Drawdown e tempo de recuperação	15%
Estabilidade entre regimes	10%
Robustez walk-forward/out-of-sample	10%
Conformidade com risco	10%
Coertura e saúde das teses	10%
Turnover, custos e capacidade	5%
Confiança dos dados e modelos	5%

Penalidades

- Dados desatualizados.
- Violação de limite.
- Concentração excessiva.
- Tese expirada.
- Posição sem evidência.
- Resultado excessivamente dependente de um regime.
- Divergência entre backtest e paper.
- Alto turnover.
- Baixa liquidez.

9.7 O que o usuário verá em cada carteira

Posição **no** ranking
Score
 Categoria
Estágio
 Retorno **no** período
 Retorno excedente
 Volatilidade
 Sharpe/Sortino
 Drawdown
 Confiança dos dados
Saúde das teses
 Última decisão **do** comitê
Próxima revisão
 Principais riscos
 Última atualização

9.8 Champion e challengers

Para cada mandato:

- Uma carteira champion.
- Até N challengers.
- Challengers não substituem automaticamente a champion.
- Promoção exige critérios previamente definidos.
- Comparação deve ocorrer em janela out-of-sample ou paper.
- Toda promoção é uma decisão de comitê.

10. Design da aplicação

Design da aplicação significa definir:

- Domínio.
- Processos.
- Estados.
- Papéis.
- Permissões.
- Regras.

- Fluxos.
- Estrutura de informação.
- Notificações.
- Auditoria.
- Critérios de decisão.

Não significa escolher cor de botão.

10.1 Personas e papéis

Administrador da plataforma

- Configura organizações.
- Gerencia usuários.
- Gerencia integrações.
- Não aprova investimento por padrão.

CIO ou responsável de investimentos

- Visualiza todas as carteiras.
- Define mandatos.
- Convoca comitês.
- Aprova estratégias.
- Consulta performance consolidada.

Gestor de carteira

- Cria propostas.
- Analisa alternativas.
- Submete rebalanceamentos.
- Não altera dados financeiros.

Analista de empresas

- Cria casos de pesquisa.
- Edita teses.
- Revisa outputs de agents.
- Aprova claims de pesquisa.

Analista macro/político

- Mantém cenários.
- Revisa eventos políticos.
- Valida impactos setoriais.

Risco

- Define limites.
- Executa stresses.
- Bloqueia propostas.
- Não altera a tese do analista.

Compliance

- Administra restricted list.
- Revisa conflitos.
- Aprova ações sensíveis.
- Consulta auditoria.

Operações

- Processa intenções aprovadas.
- Reconciliará ordens e fills futuramente.
- Não aprova a própria ordem.

Auditor

- Somente leitura.
- Acesso a versões, evidências, decisões e logs.

Viewer

- Acesso limitado a dashboards e relatórios autorizados.

10.2 Autorização

Usar RBAC mais ABAC.

RBAC define o papel.

ABAC considera:

- Organização.
- Equipe.
- Carteira.
- Estratégia.
- Classificação do dado.
- Estado do objeto.
- Valor financeiro.
- Ambiente paper/live.

Exemplo:

Um gestor pode submeter proposta em sua carteira, mas não pode aprová-la.

Risco pode rejeitar ou condicionar, mas não pode editar a tese.

Operações pode executar uma intenção aprovada, mas não pode criar ou aprovar a intenção.

11. Arquitetura de informação do painel

Navegação principal

1. Mission Control
2. Carteiras
 - 2.1 Top carteiras
 - 2.2 Carteiras-modelo
 - 2.3 Paper
 - 2.4 Live
 - 2.5 Propostas
3. Oportunidades
4. Pesquisa
 - 4.1 Asset 360
 - 4.2 Casos de pesquisa
 - 4.3 Teses
 - 4.4 Documentos
 - 4.5 Notícias e eventos
5. Macro e Política
6. Risco
7. Comitê
8. Agents
9. Qualidade dos dados
10. Backtest Lab
11. Operações
12. Auditoria e governança
13. Configurações

12. Especificação das telas

12.1 Mission Control

A tela inicial não deve ser uma coleção genérica de gráficos. Ela deve responder:

- O que mudou?
- O que requer decisão?
- Quais carteiras estão melhores?
- Quais riscos estão crescendo?

- Quais dados estão atrasados?
- Quais agents falharam?
- Que eventos podem afetar posições?

Bloco superior

- Data e hora de referência.
- Mercado aberto/fechado.
- Regime macro atual.
- Status das fontes.
- Status do sistema.
- Quantidade de alertas críticos.
- Quantidade de aprovações pendentes.

Top X carteiras

Cards ou tabela contendo:

- Ranking.
- Nome.
- Mandato.
- Score.
- Retorno excedente.
- Drawdown.
- Risco.
- Confiança.
- Estado.
- Último rebalanceamento.

Daily Intelligence Brief

Resumo produzido a partir de dados verificados:

- Mercado.
- Macro.
- Política.
- Regulação.
- Empresas da carteira.
- Oportunidades.
- Riscos.
- Agenda do dia.

Cada item abre as evidências.

Eventos críticos

- Evento.
- Entidade.
- Materialidade.
- Confiança.
- Posição afetada.
- Tese afetada.
- Ação pendente.
- SLA.

Funil de oportunidades

```
discovered
screened
researching
candidate
committee
approved
portfolio
rejected
```

Pendências

- Teses expiradas.
- Documentos não processados.

- Dados em quarentena.
 - Rebalanceamentos pendentes.
 - Limites violados.
 - Agent runs falhos.
-

12.2 Top carteiras

Filtros

- Categoria.
- Período.
- Benchmark.
- Risco.
- Stage.
- Paper/live.
- Moeda.
- Gestor.
- Confiança mínima.

Alternância de visualização

- Ranking.
- Cards.
- Matriz risco × retorno.
- Comparação.
- Histórico do ranking.

Comparação de carteiras

Selecionar até quatro carteiras e comparar:

- Performance.
 - Drawdown.
 - Volatilidade.
 - Exposição setorial.
 - Exposição fatorial.
 - Turnover.
 - Liquidez.
 - Saúde das teses.
 - Qualidade dos dados.
 - Divergência entre agents.
 - Decisões do comitê.
-

12.3 Portfolio 360

Aba Visão geral

- NAV.
- Retorno diário, mensal, anual e desde início.
- Benchmark.
- Excesso.
- Cash.
- Drawdown.
- Volatilidade.
- Status.
- Mandato.
- Última decisão.
- Próxima revisão.

Aba Posições

- Ticker.
- Empresa.
- Quantidade.

- Preço.
- Valor.
- Peso.
- Peso-alvo.
- P&L.
- Contribuição.
- Liquidez.
- Score.
- Saúde da tese.
- Último evento material.

Aba Performance

- Curva de NAV.
- Benchmark.
- Retorno acumulado.
- Retornos mensais.
- Rolling volatility.
- Rolling Sharpe.
- Drawdowns.
- Recovery periods.

Aba Atribuição

- Por ativo.
- Por setor.
- Por fator.
- Por decisão.
- Por evento.
- Allocation effect.
- Selection effect.
- Cost effect.

Aba Risco

- Concentração.
- Correlação.
- Exposição fatorial.
- VaR/CVaR.
- Stress.
- Liquidez.
- Capacidade.
- Limites.
- Violações.

Aba Teses

- Posição.
- Tese atual.
- Saúde.
- Convicção.
- Catalisadores.
- Riscos.
- Invalidações.
- Revisão.
- Cobertura de evidência.

Aba Eventos

Timeline de:

- Notícias.
- Documentos.
- Alterações de tese.
- Mudanças de peso.
- Decisões.
- Alertas.

- Proventos.
- Corporate actions.

Aba Rebalanceamento

- Alocação atual.
- Alocação proposta.
- Diff.
- Compras e vendas.
- Custos.
- Risco antes/depois.
- Cenários.
- Justificativa.
- Votos.
- Condições.

Aba Auditoria

Linha completa:

dado -> análise -> tese -> proposta -> decisão -> versão -> posição

12.4 Asset 360

Essa deve ser uma das telas mais importantes.

Cabeçalho

- Empresa.
- Ticker e classe.
- Setor.
- Cotação.
- Market cap.
- Liquidez.
- Status de cobertura.
- Posição total nas carteiras.
- Última atualização.

Investment summary

- Veredito atual.
- Convicção.
- Faixa de valor.
- Upside/downside.
- Tese em uma frase.
- Estado da tese.
- Próxima revisão.

Métricas

- Qualidade.
- Crescimento.
- Balanço.
- Caixa.
- Valuation.
- Dividendos.
- Mercado.
- Governança.
- Política/regulação.

Cada métrica deve mostrar:

- Valor.
- Unidade.
- Período.
- Variação.

- Percentil setorial.
- Fonte.
- Data de publicação.
- Qualidade.
- Fórmula.

Valuation

- Bear.
- Base.
- Bull.
- Probabilidades.
- Preço justo.
- Reverse DCF.
- Sensibilidades.
- Comparação com pares.

Tese

- Problema/oportunidade.
- Por que agora.
- Drivers.
- Catalisadores.
- Riscos.
- Invalidação.
- Horizonte.
- Evidências.
- Histórico de versões.

Notícias e eventos

Não apenas manchetes:

- Evento.
- Fato confirmado.
- Materialidade.
- Métricas afetadas.
- Impacto na tese.
- Corroboração.
- Evidências.

Política e regulação

- Propostas em tramitação.
- Atos publicados.
- Probabilidade.
- Estágio.
- Prazo.
- Sensibilidade da empresa.
- Cenários.

Pares

- Tabela comparativa.
- Distribuições.
- Percentis.
- Evolução temporal.

12.5 Oportunidades

Funil

Descoberta
Triagem
Pesquisa
Contra-tese
Valuation

Risco
Comitê
Aprovada
Rejeitada

Cada oportunidade deve mostrar

- Origem do sinal.
- Estratégia compatível.
- Score quantitativo.
- Materialidade.
- Confiança.
- Dados faltantes.
- Analista responsável.
- Agent responsável.
- Prazo.
- Próxima ação.

Origem possível

- Screener quantitativo.
- Documento.
- Notícia.
- Evento político.
- Divergência de valuation.
- Mudança de regime.
- Queda anormal.
- Mudança de fundamentos.
- Rotação setorial.
- Revisão de tese.

12.6 Macro e Política

Visão macro

- Regime.
- Selic.
- Curvas.
- Inflação.
- Câmbio.
- Crédito.
- Fiscal.
- Commodities.
- Cenários.

Legislative tracker

- Proposta.
- Tema.
- Estágio.
- Relator.
- Próximo evento.
- Probabilidade.
- Setores.
- Empresas.
- Materialidade.
- Fonte.

Matriz de exposição

evento político x setor x empresa x carteira

Timeline

Separar:

- Proposta.
- Tramitação.
- Votação.
- Sanção.
- Regulamentação.
- Vigência.
- Judicialização.

Regra visual

Toda probabilidade deve ser rotulada como:

```
Estimativa do modelo  
Estimativa humana  
Base rate histórica  
Cenário
```

Nunca exibir estimativa como fato.

12.7 Risk Center

Blocos

- Visão consolidada.
- Violações.
- Concentração.
- Liquidez.
- Fatores.
- Correlações.
- Stress.
- Drawdown.
- Risco de tese.
- Risco de dados.
- Risco de modelo.

Stress scenarios

- Juros +200 bps.
- BRL -15%.
- Petróleo -25%.
- Recessão.
- Choque fiscal.
- Regulação setorial.
- Perda de guidance.
- Evento idiossincrático.

Ações

- Reconhecer alerta.
- Abrir investigação.
- Bloquear compra.
- Solicitar redução.
- Suspende carteira.
- Criar condição para comitê.

12.8 Committee Room

Agenda

- Reunião.
- Participantes.
- Quórum.
- Pauta.
- Prazo.
- Material pendente.

Decision pack

- Resumo executivo.
- Tese.
- Contra-tese.
- Dados.
- Valuation.
- Risco.
- Política.
- Alternativas.
- Custos.
- Parecer de compliance.

Votos

Cada voto deve conter:

- Decisão.
- Justificativa.
- Condições.
- Confiança.
- Conflito declarado.
- Timestamp.
- Versão do pack.

Decisões possíveis

```
approve
approve_with_conditions
reject
request_more_information
defer
suspend
```

Ação de investimento

```
add
increase
maintain
reduce
exit
replace
watch
no_action
```

Condições

Exemplos:

- Limitar peso a 3%.
- Aguardar resultado trimestral.
- Exigir liquidez mínima.
- Reavaliar após votação.
- Stop de tese, não apenas stop de preço.

12.9 Agent Operations

Visão geral

- Runs ativos.
- Runs falhos.
- Fila.
- Custo diário.
- Latência.
- Modelo.
- Schema pass rate.
- Citation coverage.
- Human override.

- Evals recentes.

Detalhe de run

- Workflow.
- Agent.
- Prompt version.
- Model profile.
- Entrada.
- Evidências.
- Tool calls.
- Output estruturado.
- Guardrails.
- Erros.
- Tokens.
- Custo.
- Trace.
- Aprovações.
- Retry.

Ações

- Cancelar.
- Reexecutar.
- Reexecutar com modelo diferente.
- Marcar output incorreto.
- Abrir incidente.
- Adicionar a dataset de avaliação.

12.10 Data Quality Center

Indicadores

- Freshness por fonte.
- Compleitude.
- Taxa de parse.
- Reconciliação.
- Mapeamentos pendentes.
- Restatements.
- Quarentena.
- Duplicidade.
- SLA.

Incidente

- Fonte.
- Objeto.
- Severidade.
- Registros afetados.
- Métricas afetadas.
- Teses afetadas.
- Carteiras afetadas.
- Responsável.
- Workaround.
- Resolução.

12.11 Backtest Lab

Configuração imutável

- Estratégia.
- Universo.
- Janela.
- Calendário.

- Frequência.
- Dados point-in-time.
- Custos.
- Slippage.
- Impostos.
- Liquidez.
- Benchmark.
- Rebalanceamento.
- Modelo.
- Prompt versions.
- Seed.

Resultados

- Performance.
- Drawdown.
- Risco.
- Regimes.
- Sensibilidade.
- Walk-forward.
- Out-of-sample.
- Turnover.
- Capacity.
- Baselines.
- Intervalos de confiança.

Badges

PIT verified

Corporate actions verified

Survivorship-bias checked

Costs included

Out-of-sample

Reproducible

13. Design do frontend

13.1 Direção visual

O produto deve parecer uma plataforma institucional de pesquisa e risco, não um aplicativo de aposta ou day trade.

Características:

- Alta densidade informacional.
- Hierarquia rigorosa.
- Pouca decoração.
- Dados com contexto.
- Cores semânticas.
- Evidência sempre acessível.
- Atualização e qualidade visíveis.
- Gráficos comparáveis.
- Estados de erro explícitos.

13.2 Layout

Desktop

- Target principal: 1440 px ou superior.
- Sidebar: 240 px expandida, 72 px recolhida.
- Topbar: 64 px.
- Grid: 12 colunas.
- Espaçamento base: 8 px.
- Conteúdo máximo: 1600 px.
- Painéis redimensionáveis apenas onde agregam valor.

Tablet

- Sidebar recolhida.
- Tabelas com colunas prioritárias.
- Painéis secundários em drawers.

Mobile

Mobile não deve tentar reproduzir toda a estação de trabalho.

Escopo mobile:

- Monitorar.
- Receber alertas.
- Ler resumo.
- Aprovar ou rejeitar quando permitido.
- Consultar posições.

13.3 Temas e tokens

Tema claro

canvas	#F5F7FA
surface	#FFFFFF
surface-muted	#EEF2F6
border	#D7DEE8
text-primary	#122033
text-secondary	#5E6B7A

Tema escuro

canvas	#0B1220
surface	#111B2E
surface-muted	#18243A
border	#27344A
text-primary	#E8EEF6
text-secondary	#99A8BA

Cores semânticas

brand	#2563EB
positive	#15803D
negative	#B91C1C
warning	#B45309
info	#0369A1
critical	#991B1B

Nunca depender somente de vermelho e verde. Sempre adicionar:

- Ícone.
- Texto.
- Sinal.
- Padrão.
- Tooltip.

13.4 Tipografia

- Interface: Inter ou IBM Plex Sans.
- Números e código: JetBrains Mono.
- Usar `font-variant-numeric: tabular-nums`.
- Evitar fontes excessivamente condensadas.
- Hierarquia limitada e consistente.

13.5 Componentes específicos do domínio

- PortfolioRankCard
- MandateBadge
- StageBadge
- EvidenceBadge
- DataFreshnessBadge

- DataQualityIndicator
- ConfidenceBreakdown
- ThesisHealthMeter
- RiskLimitChip
- MaterialityBadge
- FactInferenceRecommendationTag
- SourceDrawer
- DecisionTimeline
- PortfolioDiff
- ScenarioWaterfall
- AgentRunTimeline
- ApprovalPanel
- PolicyStageTracker
- MetricProvenancePopover
- AsOfIndicator

13.6 Regras para gráficos

- Nunca usar 3D.
- Benchmark sempre claramente identificado.
- Incluir bandas de incerteza quando aplicável.
- Mostrar data de referência.
- Mostrar fonte.
- Mostrar unidade.
- Mostrar se o dado é nominal, real, anualizado ou acumulado.
- Evitar dois eixos Y quando possível.
- Tooltips devem usar precisão definida por tipo de métrica.
- Drawdown sempre negativo.
- Retorno e risco não devem compartilhar escala sem indicação.
- Cenários devem apresentar probabilidades separadas de resultados.
- Gráficos políticos devem distinguir estágio de probabilidade.

13.7 Estados obrigatórios

Toda tela deve prever:

- Loading.
- Empty.
- Partial data.
- Stale data.
- Quarantined data.
- No permission.
- Source unavailable.
- Model unavailable.
- Calculation failed.
- No evidence.
- Conflicting evidence.
- Awaiting approval.

Não exibir zero quando o estado correto for “indisponível”.

13.8 Stack frontend

Recomendação:

```
Next.js
React
TypeScript
TanStack Query
TanStack Table
Radix UI
Tailwind CSS ou CSS tokens próprios
ECharts
React Hook Form
Zod
OpenAPI-generated client
Storybook
Vitest
```

```
Testing Library
Playwright
axe-core
```

Decisões

- SSE para atualizações de workflow e notificações.
- WebSocket somente quando comunicação bidirecional contínua for necessária.
- Filtros persistidos na URL.
- Feature flags.
- Client gerado pelo OpenAPI.
- Nenhuma chave sensível no navegador.
- Valores monetários tratados como decimal/string, não `number` indiscriminadamente.
- Datas apresentadas no timezone da organização.
- Localização inicial `pt-BR`, mas domínio preparado para internacionalização.

14. Arquitetura técnica-alvo

14.1 Estratégia: monólito modular com deployables separados

Não começar com dezenas de microserviços.

Usar:

```
Monólito modular de domínio
+
workers separados por carga e permissão
+
frontend separado
```

14.2 Planos do sistema

Data Plane

- Coleta.
- Raw Zone.
- Parsing.
- Normalização.
- Qualidade.
- Fatos.
- Métricas.

Intelligence Plane

- RAG.
- Agents.
- Notícias.
- Política.
- Macro.
- Pesquisa.
- Teses.

Decision Plane

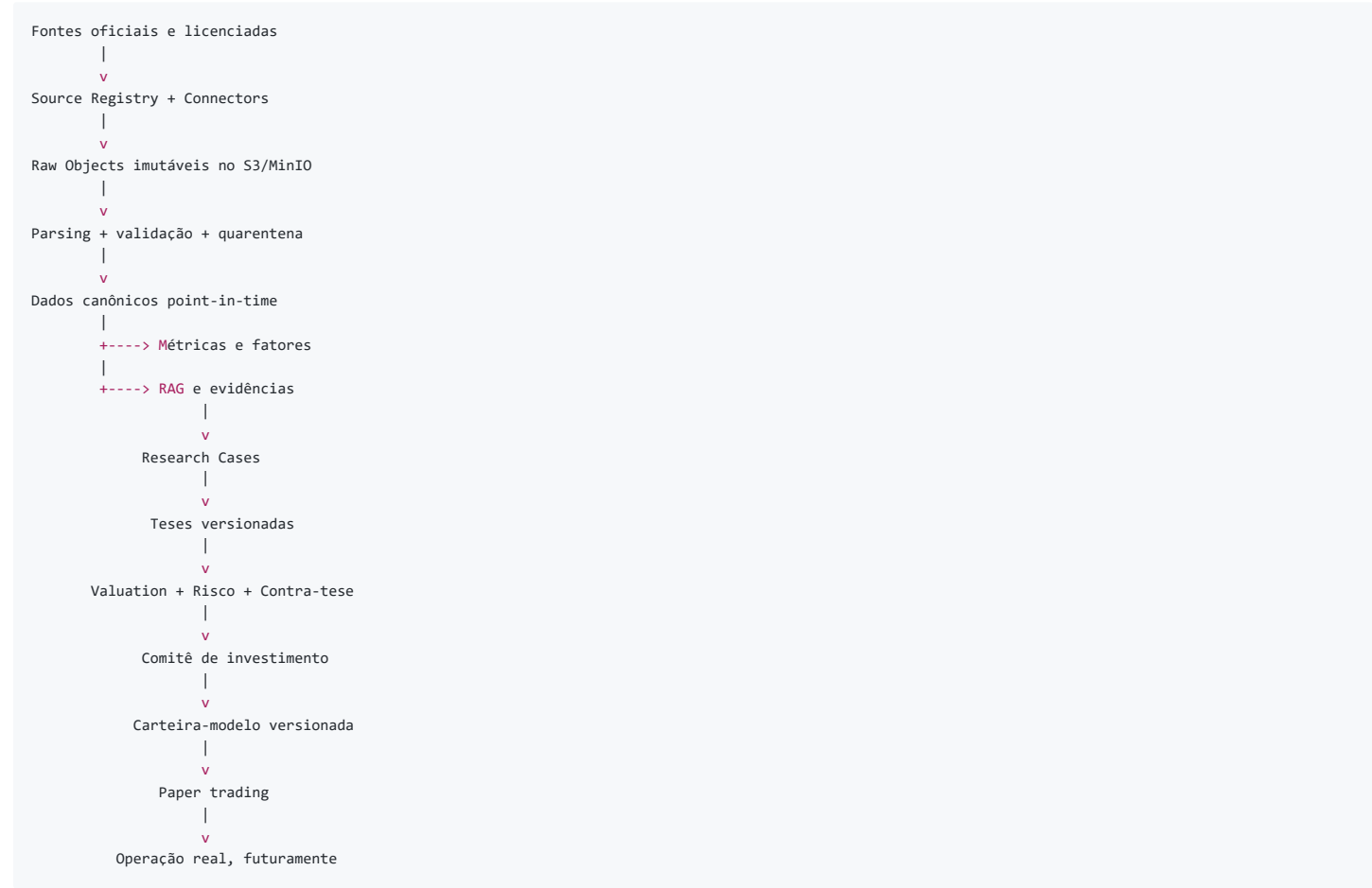
- Valuation.
- Risco.
- Otimização.
- Comitê.
- Carteiras.
- Rebalanceamento.

Control Plane

- Temporal.
- Segurança.
- Policies.

- Observabilidade.
- Evals.
- Auditoria.
- Custos.

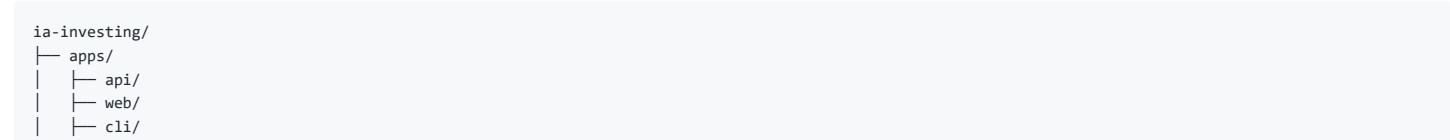
14.3 Fluxo



14.4 Bounded contexts



14.5 Estrutura do repositório




```
|   └─ workers/
|       └─ data/
|       └─ research/
|       └─ portfolio/
|       └─ notifications/
└─ packages/
    └─ ia_investing/
        └─ identity/
        └─ instruments/
        └─ sources/
        └─ ingestion/
        └─ documents/
        └─ financials/
        └─ market_data/
        └─ news/
        └─ macro/
        └─ politics/
        └─ research/
        └─ thesis/
        └─ valuation/
        └─ scoring/
        └─ portfolio/
        └─ risk/
        └─ committee/
        └─ execution/
        └─ ai/
        └─ evaluation/
        └─ governance/
        └─ platform/
            └─ database/
            └─ temporal/
            └─ telemetry/
            └─ storage/
            └─ security/
            └─ messaging/
└─ prompts/
└─ migrations/
    └─ versions/
└─ tests/
    └─ unit/
    └─ integration/
    └─ contract/
    └─ workflow/
    └─ replay/
    └─ backtest/
    └─ evals/
    └─ golden/
    └─ e2e/
└─ infra/
    └─ compose/
    └─ terraform/
    └─ kubernetes/
    └─ observability/
└─ docs/
    └─ adr/
    └─ architecture/
    └─ domain/
    └─ product/
    └─ runbooks/
    └─ security/
```

15. Modelo de dados-alvo

15.1 Identity

```
organizations
users
teams
memberships
roles
permissions
role_permissions
user_roles
api_clients
sessions
```

15.2 Instrument master

```
legal_entities
issuers
```

```
instruments
listings
ticker_history
instrument_identifiers
sectors
industries
peer_groups
issuer_aliases
corporate_relationships
```

O ticker não é o ativo. Ticker é uma identificação temporal de uma listagem.

15.3 Source catalog

```
source_registry
source_credentials
source_license
source_rate_limit
source_schema_version
source_sla
source_health
```

15.4 Raw e ingestão

```
ingestion_run
source_object
source_object_version
download_attempt
parser_run
schema_validation
quarantine_record
lineage_edge
```

15.5 Financial facts

```
reporting_period
financial_taxonomy
financial_account
account_mapping_version
account_mapping_rule
financial_fact
financial_fact_revision
metric_definition
metric_definition_version
metric_value
metric_lineage
restatement
```

15.6 Market data

```
market_bar
market_quote
corporate_action
dividend
interest_on_equity
split
subscription
buyback
fx_rate
interest_curve
commodity_price
index
index_constituent
trading_calendar
```

15.7 Notícias e política

```
content_source
content_item
content_version
entity_mention
source_claim
detected_event
event_source
event_corroboration
```

```
event_impact
policy_proposal
policy_stage
policy_actor
policy_vote
regulatory_action
sector_policy_exposure
issuer_policy_exposure
```

15.8 Pesquisa e tese

```
research_case
research_assignment
research_question
evidence
claim
claim_evidence
assessment
investment_thesis
thesis_version
thesis_catalyst
thesis_risk
thesis_invalidation_rule
valuation_model
valuation_scenario
peer_comparison
recommendation
```

15.9 Agents

```
agent_definition
agent_version
prompt_template
prompt_version
model_profile
tool_definition
tool_policy
agent_run
agent_run_input
agent_run_output
agent_tool_call
agent_approval
agent_feedback
evaluation_dataset
evaluation_case
evaluation_run
model_budget
```

15.10 Portfólio

```
strategy_mandate
model_portfolio
portfolio_version
portfolio_benchmark
portfolio_nav
position_snapshot
cash_snapshot
portfolio_transaction
rebalance_proposal
proposed_trade
trade_intent
order
fill
fee
tax
reconciliation
```

15.11 Risco

```
risk_policy
risk_limit
risk_model
risk_model_version
risk_snapshot
factor_exposure
liquidity_snapshot
stress_scenario
stress_result
```

```
risk_breach  
risk_waiver
```

15.12 Comitê

```
committee  
committee_member  
committee_meeting  
agenda_item  
decision_pack  
committee_vote  
committee_decision  
decision_condition  
decision_signature
```

15.13 Governança

```
audit_event  
policy  
policy_version  
incident  
exception_request  
restricted_list  
conflict_declaration  
model_registry  
data_retention_policy
```

16. Temporalidade e versionamento

Todo dado relevante deve considerar:

```
effective_at  
published_at  
discovered_at  
ingested_at  
validated_at  
knowledge_at  
valid_from  
valid_to  
revision_number
```

Exemplo

Um resultado do quarto trimestre pode:

- Referir-se a 31 de dezembro.
- Ser publicado em março.
- Ser descoberto cinco minutos depois.
- Ser validado vinte minutos depois.
- Ser reapresentado em maio.

O backtest de janeiro não pode usar esse resultado.

A consulta correta é:

```
Quais fatos o sistema conhecia em 15 de janeiro às 18h?
```

e não:

```
Quais fatos hoje possuem period_end em dezembro?
```

17. Pipeline político e regulatório

17.1 Descoberta

- Novas proposições.
- Mudança de tramitação.

- Nomeação de relator.
- Entrada em pauta.
- Parecer.
- Votação.
- Sanção ou veto.
- Regulamentação.
- Publicação no DOU.
- Decisão judicial ou administrativa relevante.

17.2 Normalização

- Identificar o objeto jurídico.
- Vincular versões.
- Vincular autores e relatores.
- Identificar temas.
- Vincular setores.
- Identificar prazos.

17.3 Avaliação quantitativa

- Base rate de avanço por tipo.
- Estágio.
- Apoio.
- Urgência.
- Histórico de votações.
- Calendário.
- Eventos impeditivos.
- Probabilidade com intervalo.

17.4 Avaliação qualitativa

O agent analisa:

- Conteúdo material.
- Alterações entre versões.
- Mecanismo econômico.
- Winners e losers.
- Timing.
- Risco de judicialização.
- Necessidade de regulamentação posterior.

17.5 Mapeamento para investimentos

```
policy_event
-> setor
    -> driver operacional
        -> métrica financeira
            -> empresa
                -> tese
                    -> carteira
```

Exemplo:

```
Mudança tarifária
-> energia
-> receita permitida
-> EBITDA e FCF
-> empresas do setor
-> valuation
-> posições das carteiras
```

18. Scorecards de ações

O scorecard atual recebe pilares já normalizados entre zero e um, mas não define como essa normalização acontece. Também existem exclusões configuradas com nomes de métricas enquanto o código compara nomes de pilares, o que impede a exclusão pretendida. A ausência de dados repondera automaticamente os demais pilares, podendo produzir scores altos com informação insuficiente. ([GitHub](#))

Novo modelo

Etapa 1 — Métricas brutas

Valores reais e auditados.

Etapa 2 — Transformações

- Winsorization.
- Z-score.
- Percentil.
- Ajuste setorial.
- Ajuste temporal.
- Direção da métrica.
- Penalidade por qualidade.

Etapa 3 — Pilares

```
quality
growth
balance_sheet
cash_generation
valuation
momentum
dividend
governance
event_risk
macro_sensitivity
```

Etapa 4 — Score de estratégia

Cada estratégia possui pesos próprios.

Etapa 5 — Confiança

```
effective_score =
base_score
× data_quality_factor
× coverage_factor
× thesis_freshness_factor
```

Etapa 6 — Vetoos separados

Veto não deve apenas reduzir score para 0,2.

Ele deve gerar:

```
eligibility = blocked
reason = ...
```

Scorecards específicos

- Industrial.
- Banco.
- Seguradora.
- Utility.
- Commodity.
- Varejo.
- Construção.
- Tecnologia.
- Real estate.
- Dividendos.
- Turnaround.
- Small caps.

Um banco não deve ser avaliado por current ratio como uma indústria.

19. Valuation

19.1 Resultados

Não retornar um único “preço justo”.

Retornar:

```
bear
base
bull
probability-weighted
reverse_dcf
relative_valuation
market_implied_scenario
```

19.2 Cada cenário

- Receita.
- Margem.
- Capex.
- Capital de giro.
- WACC.
- Crescimento terminal.
- Dívida.
- Ações.
- Horizonte.
- Probabilidade.
- Fonte de premissas.
- Sensibilidades.

19.3 Agents

Agents podem propor premissas, mas:

- Fórmulas são executadas em código.
- Premissas precisam de evidência.
- Mudanças precisam de diff.
- Valuation precisa ser reproduzível.
- Probabilidades precisam ser aprovadas.

20. Motor de risco e otimização

20.1 Função objetivo

Exemplo:

$$[\max_w ; \mu^{\text{top } w} - \lambda w^{\text{top } \Sigma} w - \gamma \text{Vert } w - w_0 \text{rVert}_1 - \eta C(w) - \rho P(w)]$$

Onde:

- (w): pesos propostos.
- (w_0): pesos atuais.
- (μ): retornos esperados por cenário.
- (Σ): matriz de risco.
- ($C(w)$): custos.
- ($P(w)$): penalidades de concentração ou fatores.

20.2 Restrições

- Soma dos pesos.
- Caixa mínimo e máximo.
- Peso máximo e mínimo.
- Setor.
- Fator.
- Liquidez.
- Turnover.
- Capacidade.

- Lote.
- Restricted list.
- Tese ativa.
- Convicção mínima.
- Data quality.
- Políticas específicas do mandato.

20.3 Retorno esperado

Não usar diretamente:

```
LLM diz que retorno será 20%
```

Usar:

```
retorno esperado =  
soma(probabilidade do cenário × retorno do cenário)
```

Probabilidade e retorno devem possuir fontes e versão.

20.4 Resultado

```
status  
weights  
trades  
expected_return  
expected_volatility  
expected_drawdown  
factor_exposure  
liquidity_usage  
turnover  
costs  
constraint_slacks  
binding_constraints  
solver  
solver_version  
input_snapshot_id
```

21. Backtesting correto

21.1 Requisitos

- Universo histórico.
- Dados point-in-time.
- Datas de publicação.
- Reapresentações.
- Delistings.
- IPOs.
- Mudanças de ticker.
- Corporate actions.
- Dividendos.
- JCP.
- Custos.
- Slippage.
- Impostos.
- Liquidez.
- Capacidade.
- Calendário.
- Delay de execução.
- Benchmark fora do universo.
- Cash.
- Walk-forward.
- Out-of-sample.

21.2 Testes anti-look-ahead

- Remover dados publicados depois da data.
- Alterar um fato futuro e verificar que o passado não muda.
- Confirmar que composição histórica do universo é respeitada.
- Confirmar que reapresentação não substitui silenciosamente a versão original.
- Confirmar que preço de execução ocorre depois do sinal.
- Confirmar que benchmark não recebe peso.

21.3 Comparações

- Benchmark.
- Equal weight.
- Ranking quantitativo sem agents.
- Estratégia sem notícias.
- Estratégia sem política.
- Estratégia sem LLM.
- Carteira anterior.

Isso permite medir o valor incremental dos agents.

22. Workflows-alvo

22.1 SourceIngestionWorkflow

Gatilho

Schedule ou evento de fonte.

Passos

1. Verificar source registry.
 2. Aplicar rate limit.
 3. Descobrir objetos.
 4. Comparar ETag/hash.
 5. Baixar.
 6. Preservar original.
 7. Registrar versão.
 8. Publicar evento.
 9. Atualizar freshness.
 10. Encerrar idempotentemente.
-

22.2 FilingPublishedWorkflow

1. Receber documento.
 2. Confirmar emissor.
 3. Extrair fatos.
 4. Normalizar.
 5. Validar.
 6. Quarentenar inconsistências.
 7. Calcular métricas.
 8. Comparar períodos.
 9. Executar Filing Analyst.
 10. Executar Forensic Agent.
 11. Executar Critic.
 12. Identificar teses afetadas.
 13. Abrir revisão.
 14. Calcular impacto nas carteiras.
 15. Notificar.
-

22.3 NewsEventWorkflow

1. Ingerir conteúdo.

2. Canonicalizar URL.
3. Calcular hash.
4. Deduplicar.
5. Resolver entidades.
6. Classificar fonte.
7. Extrair claims.
8. Corroborar.
9. Classificar evento.
10. Calcular materialidade.
11. Comparar com métricas.
12. Comparar com tese.
13. Abrir alerta ou arquivar.
14. Atualizar timeline.

22.4 PolicyEventWorkflow

1. Detectar proposta ou ato.
2. Normalizar identidade.
3. Comparar versão.
4. Atualizar estágio.
5. Mapear temas.
6. Atualizar atores.
7. Estimar base rate.
8. Executar Political Analyst.
9. Mapear setores e emissores.
10. Rodar cenários.
11. Identificar teses e carteiras.
12. Solicitar revisão humana quando material.

22.5 DailyMarketCloseWorkflow

1. Validar fechamento.
2. Ajustar corporate actions.
3. Calcular preços oficiais.
4. Atualizar NAV.
5. Calcular P&L.
6. Atualizar benchmark.
7. Calcular attribution.
8. Calcular risco.
9. Verificar limites.
10. Atualizar ranking.
11. Produzir relatório diário.
12. Reconciliar paper/live.

22.6 OpportunityDiscoveryWorkflow

1. Construir universo elegível.
2. Aplicar filtros.
3. Calcular fatores.
4. Detectar mudanças.
5. Detectar eventos.
6. Combinar sinais.
7. Criar candidatos.
8. Remover duplicados.
9. Executar triagem qualitativa.
10. Criar research cases.
11. Priorizar fila.

22.7 ThesisReviewWorkflow

1. Carregar versão ativa.
2. Carregar novos fatos.
3. Executar especialistas.
4. Verificar contradições.
5. Recalcular valuation.
6. Atualizar riscos.
7. Gerar diff.
8. Solicitar aprovação.
9. Criar nova versão.
10. Invalidar ou manter recomendação.

22.8 PortfolioConstructionWorkflow

1. Carregar mandato.
2. Carregar universo aprovado.
3. Verificar elegibilidade.
4. Construir retornos por cenário.
5. Construir risco.
6. Executar otimização.
7. Validar constraints.
8. Comparar alternativas.
9. Gerar proposta.
10. Executar Risk Director.
11. Executar Compliance.
12. Enviar para comitê.

22.9 InvestmentCommitteeWorkflow

1. Congelar decision pack.
2. Verificar quórum.
3. Coletar pareceres.
4. Coletar votos.
5. Verificar conflitos.
6. Consolidar decisão.
7. Registrar condições.
8. Assinar.
9. Criar versão aprovada.
10. Notificar interessados.

22.10 RebalanceWorkflow

1. Receber versão aprovada.
2. Calcular diff.
3. Simular custos.
4. Verificar preços e liquidez.
5. Criar trade intents.
6. Solicitar aprovações.
7. Executar em paper.
8. Reconciliar.
9. Atualizar posições.
10. Registrar slippage.

23. Contrato da API

23.1 Queries

```
GET /v1/dashboard/mission-control
GET /v1/model-portfolios
GET /v1/model-portfolios/{id}
```

```
GET /v1/model-portfolios/{id}/versions
GET /v1/model-portfolios/{id}/nav
GET /v1/model-portfolios/{id}/positions
GET /v1/assets/{instrument_id}/overview
GET /v1/assets/{instrument_id}/metrics
GET /v1/assets/{instrument_id}/thesis
GET /v1/assets/{instrument_id}/events
GET /v1/research/cases
GET /v1/policy/events
GET /v1/risk/breaches
GET /v1/committee/meetings
GET /v1/agent-runs
GET /v1/data-quality/incidents
GET /v1/audit/events
```

23.2 Commands

```
POST /v1/research/cases
POST /v1/research/cases/{id}/submit
POST /v1/theses/{id}/revisions
POST /v1/portfolios/{id}/rebalance-proposals
POST /v1/committee/meetings
POST /v1/committee/decisions/{id}/votes
POST /v1/approvals/{id}/approve
POST /v1/approvals/{id}/reject
POST /v1/agent-runs
POST /v1/backtests
```

23.3 Regras

- Comandos longos: 202 Accepted .
- Idempotency-Key obrigatório.
- ETag e If-Match para concorrência.
- Cursor pagination.
- Problem Details para erros.
- Correlation ID.
- as_of explícito em consultas temporais.
- Campos monetários como decimal serializado.
- Schemas de resposta dedicados.
- Nunca retornar diretamente ORM.

23.4 Streaming

```
GET /v1/streams/operations/{operation_id}
GET /v1/streams/notifications
```

SSE inicialmente.

24. Segurança e governança

24.1 Controles

- OIDC/OAuth2.
- MFA.
- RBAC + ABAC.
- Segregação de funções.
- Regra de quatro olhos.
- Secrets Manager/Vault.
- TLS.
- Criptografia em repouso.
- Rotação de credenciais.
- Egress allowlist.
- Rate limiting.
- Audit append-only.
- Backups testados.
- Restauração testada.
- Kill switch.

- Retention policy.
- Restricted list.
- Conflict declaration.
- Threat modeling.
- Dependency scanning.
- Secret scanning.
- SAST.
- Container scanning.

24.2 Agents

- Identidade própria por agent.
- Ferramentas mínimas.
- Read-only por default.
- Sem SQL arbitrário.
- Sem shell.
- Sem corretora.
- Sem credencial exposta em prompt.
- Limite de custo.
- Limite de turnos.
- Domínios permitidos.
- Input tratado como não confiável.
- Output validado antes de persistir.

24.3 Auditoria

O evento de auditoria deve ser append-only:

```
actor_type
actor_id
action
resource_type
resource_id
before_hash
after_hash
reason
correlation_id
ip
user_agent
occurred_at
```

Para decisões críticas, adicionar assinatura lógica ou hash encadeado.

25. Perímetro regulatório

O produto precisa distinguir claramente quatro situações:

1. Ferramenta interna de pesquisa própria.
2. Produção e distribuição de relatórios.
3. Consultoria individualizada.
4. Administração ou execução de carteira.

A Resolução CVM 19 trata da consultoria de valores mobiliários; a Resolução CVM 20 trata da atividade de analista; e a Resolução CVM 21 trata da administração profissional de carteiras. ([CVM](#))

Antes de oferecer recomendações a terceiros, personalizar decisões para clientes ou administrar recursos, o produto precisa de avaliação jurídica especializada.

Workstream regulatório

- Definir finalidade interna ou comercial.
- Definir quem receberá relatórios.
- Definir se há individualização.
- Definir se o sistema apenas recomenda ou executa.
- Definir responsabilidades profissionais.
- Registrar conflitos.
- Registrar aprovação e autoria.
- Criar disclosures.

- Controlar uso de dados licenciados.
 - Definir retenção.
 - Definir suitability quando aplicável.
 - Definir trilha de revisão humana.
-

26. Observabilidade

26.1 Métricas técnicas

- Requests por endpoint.
- Latência p50/p95/p99.
- Taxa de erro.
- Conexões de banco.
- Locks.
- Tempo de query.
- Tamanho das filas.
- Workflow age.
- Activity retries.
- Worker saturation.
- Uso de memória e CPU.
- Disponibilidade de fontes.

26.2 Métricas de dados

- Freshness.
- Completeness.
- Validade.
- Unicidade.
- Reconciliação.
- Parse rate.
- Quarentena.
- Restatements.
- Mapeamentos desconhecidos.
- Lineage coverage.

26.3 Métricas de IA

- Runs.
- Taxa de sucesso.
- Schema pass rate.
- Citation coverage.
- Claims suportados.
- Hallucination findings.
- Guardrail trips.
- Tokens.
- Custo.
- Latência.
- Modelo.
- Prompt version.
- Human override.
- Divergência entre agents.

26.4 Métricas de negócio

- Teses ativas.
- Teses expiradas.
- Cobertura do universo.
- Oportunidades por estágio.
- Tempo até decisão.
- Aprovação/rejeição.
- Violações.
- Performance.

- Drawdown.
- Turnover.
- Valor incremental dos agents.
- Divergência entre proposta e execução.

26.5 SLOs iniciais propostos

Capacidade	SLO inicial
API de leitura	99,9% de disponibilidade
Workflows críticos	99,5% concluídos dentro da janela
Ingestão CVM	95% até 30 minutos após disponibilidade detectável
Evento crítico	95% classificado dentro da janela da fonte
NAV diário	Publicado após reconciliação até horário operacional definido
Citation coverage	100% para claims materiais
Dados críticos sem origem	0
Ordem sem aprovação	0
Violação crítica ignorada	0

27. Estratégia de testes

27.1 Unitários

- Fórmulas.
- Transformações.
- State machines.
- Policies.
- Mapeamentos.
- Parsers pequenos.

27.2 Property-based

- Ativo = Passivo + PL dentro da tolerância.
- Pesos somam 100%.
- Limites nunca são ultrapassados.
- NAV preserva identidade contábil.
- Reexecutar activity não duplica resultado.
- Backtest não acessa datas futuras.

27.3 Integração

- PostgreSQL real.
- MinIO real.
- Temporal test server.
- Migrations.
- API.
- Outbox.
- RAG.

27.4 Contract tests

Um conjunto por fonte:

- CVM.
- B3.
- BCB.
- Câmara.
- Senado.
- DOU.
- RI.

- Notícias.

Salvar fixtures representativas sem redistribuir conteúdo sem licença.

27.5 Workflow tests

- Happy path.
- Retry.
- Timeout.
- Cancel.
- Pause.
- Aprovação.
- Quarentena.
- Replay após mudança de código.

27.6 Agent evals

- Extração.
- Classificação.
- Claims.
- Citações.
- Materialidade.
- Comparação com tese.
- Calibração de confiança.
- Robustez a prompt injection.
- Custo.
- Latência.
- Decisão.

27.7 Frontend

- Componentes no Storybook.
- Visual regression.
- Acessibilidade.
- Teclado.
- Responsividade.
- E2E de fluxos críticos.
- Erros e estados vazios.

27.8 CI

Gate mínimo:

```
ruff
mypy
pytest
coverage
alembic check
migration dry-run
OpenAPI diff
schema compatibility
Temporal replay
agent eval threshold
frontend lint
frontend tests
Playwright
dependency scan
secret scan
container scan
```

28. Roadmap de execução

Fase 0 — Congelamento e baseline

Objetivo

Criar segurança para modificar o código.

Entregas

- Tag do estado atual.
- ADRs iniciais.
- Inventário de módulos.
- Inventário de schemas.
- Inventário de fontes.
- Fixtures CVM/B3.
- CI básico.
- Relatório de dependências.
- Backlog P0/P1/P2.
- Convenções de domínio.

Aceite

- Branch protegido.
- CI obrigatório.
- Testes atuais executados.
- Métricas de baseline registradas.

Fase 1 — Tornar o sistema executável

Escopo

- Renomear namespace.
- Corrigir dependências.
- Unificar settings.
- Criar migration inicial.
- Corrigir mapeamentos SQLAlchemy.
- Criar activities.
- Registrar workers.
- Substituir scheduler por Temporal Schedules.
- Corrigir prompt paths.
- Alinhar schemas.
- Completar Docker Compose.
- Integrar telemetry básica.

Aceite

Um cenário de teste deve:

1. Subir toda a infraestrutura.
2. Aplicar migrations.
3. Ingerir fixture CVM.
4. Preservar raw.
5. Extrair fatos.
6. Validar.
7. Calcular métricas.
8. Executar agent mockado.
9. Persistir análise.
10. Expor resultado na API.

Fase 2 — Confiança dos dados

Escopo

- Source Registry.
- Raw object versioning.
- Account taxonomy.
- Financial facts.
- Point-in-time.
- Reapresentações.
- DFC e DVA.
- Quarentena.

- Data quality incidents.
- Linhagem.
- Corporate actions.
- Trading calendar.

Aceite

- Amostra multi-setorial reconciliada.
- Parse error nunca vira zero.
- Cada métrica aponta para fatos de origem.
- Consulta `as_of` reproduz estado histórico.

Fase 3 — Domínio de pesquisa

Escopo

- Research cases.
- Evidence.
- Claims.
- Teses e versões.
- Catalisadores.
- Riscos.
- Regras de invalidação.
- Valuation scenarios.
- Revisão humana.
- Timeline.

Aceite

É possível explicar completamente:

`Por` que `PETR4` possui recomendação `X` em uma `data` específica?

com evidências, versões e responsáveis.

Fase 4 — Framework de agents

Escopo

- Agent registry.
- Agent versions.
- Prompt versions.
- Model profiles.
- Typed tools.
- Guardrails.
- Budgets.
- Evals.
- HITL.
- Tracing.
- Filing, news, macro, political e critic.

Aceite

- Todos os outputs validam schema.
- Claims materiais têm evidência.
- Mudança de modelo exige eval.
- Tool sensível exige aprovação.
- Custo por caso é mensurável.

Fase 5 — Carteiras, risco e backtest

Escopo

- Mandatos.
- Carteiras-modelo.
- Versions.
- Position snapshots.
- NAV.
- Benchmarks.
- Risk policies.
- Risk snapshots.
- Stress.
- Otimizador corrigido.
- Backtest PIT.
- Paper execution.

Aceite

- Carteira reproduzível por data.
- NAV reconciliado.
- Proposta respeita todas as restrições.
- Solver não retorna fallback inválido.
- Backtest passa nos testes anti-look-ahead.

Fase 6 — Painel MVP

Escopo

- Design system.
- App shell.
- Mission Control.
- Top carteiras.
- Portfolio 360.
- Asset 360.
- Oportunidades.
- Risco.
- Comitê.
- Agents.
- Data Quality.

Aceite

Um usuário autorizado consegue:

1. Ver uma oportunidade.
2. Abrir o caso.
3. Ler evidências.
4. Revisar tese.
5. Consultar valuation.
6. Consultar risco.
7. Avaliar proposta.
8. Votar no comitê.
9. Ver carteira atualizada.
10. Auditar o caminho completo.

Fase 7 — Inteligência política e macro completa

Escopo

- Câmara.
- Senado.
- DOU.
- Reguladores.
- Policy graph.
- Probabilidades.
- Setor/empresa/carteira.

- Cenários.
- Dashboard político.

Aceite

Um evento regulatório pode ser rastreado desde a fonte até o impacto previsto em métricas, teses e carteiras.

Fase 8 — Operação paper institucional

Escopo

- Simulação de ordens.
- Fills.
- Slippage.
- Custos.
- Reconciliação.
- Alertas.
- Post-mortem.
- Champion/challenger.

Aceite

- Carteira paper opera sem intervenção técnica manual.
- Toda negociação deriva de versão aprovada.
- Divergências são reconciliadas.
- Resultados são atribuídos a decisões.

Fase 9 — Produção controlada e eventual execução real

Somente depois de:

- Revisão jurídica.
- Dados licenciados.
- Segurança auditada.
- Segregação de funções.
- Kill switch.
- Reconciliação.
- Limites.
- Disaster recovery.
- Paper trading estável.
- Model risk governance.
- Aprovação formal.

29. Primeiros pull requests, em ordem

PR-001 — Namespace e imports

- Criar `ia_investing`.
- Mover módulos.
- Eliminar colisão `agents`.
- Corrigir imports.
- Adicionar import tests.

PR-002 — Dependências e configuração

- Declarar `asynpg`.
- Criar lockfile.
- Unificar `.env`.
- Separar settings.
- Validar produção.

PR-003 — Banco e migration baseline

- Converter modelos para `Mapped` .
- Corrigir JSONB.
- Criar naming convention.
- Criar migration inicial.
- Remover `create_all` .

PR-004 — Contratos canônicos

- Consolidar Pydantic schemas.
- Eliminar dataclasses duplicadas.
- Versionar schemas.
- Testar serialização.

PR-005 — Temporal workers e activities

- Implementar activities.
- Registrar por task queue.
- Configurar retries.
- Configurar idempotência.
- Criar workflow tests.

PR-006 — Temporal Schedules

- Remover scheduler in-memory.
- Criar schedules.
- Configurar overlap/catch-up.
- Expor status.

PR-007 — Agent runtime v2

- Agent Registry.
- Prompt loader.
- Model profiles.
- Structured outputs.
- Tool policies.
- Tracing.
- Mock provider.

PR-008 — Infra local completa

- API.
- Workers.
- Migration job.
- MinIO init.
- MLflow DB.
- Telemetry stack.
- Healthchecks.

PR-009 — Source registry e Raw Zone

- Sources.
- Licenses.
- Rate limits.
- Source objects.
- Versions.
- Hashes.
- Quarentena.

PR-010 — CVM financial facts

- Taxonomia.
- Mapeamento.
- DFC/DVA.
- Reapresentação.

- Value status.
- Fixtures oficiais.

PR-011 — Research domain

- Cases.
- Evidence.
- Claims.
- Thesis versions.
- State machine.
- Audit.

PR-012 — Identity e autorização

- OIDC.
- Roles.
- Permissions.
- Organization.
- Audit context.

PR-013 — Portfolio domain

- Mandate.
- Versions.
- Snapshots.
- NAV.
- Benchmark.
- Proposals.

PR-014 — Web shell e design system

- Next.js.
- Tokens.
- App shell.
- Auth.
- Generated client.
- Storybook.

PR-015 — Mission Control e Top carteiras

- Agregações.
- Ranking.
- Filtros.
- Comparação.
- Freshness.
- Alerts.

30. Definition of Done

Nenhuma feature é concluída apenas porque “o endpoint funciona”.

Cada entrega deve possuir:

- Requisito de negócio.
- Modelo de domínio.
- Migration.
- Serviço de aplicação.
- Policy de autorização.
- Audit event.
- Telemetria.
- Testes unitários.
- Testes integrados.
- Documentação.
- Tratamento de erro.

- Estado vazio.
 - Estado de dado parcial.
 - Runbook.
 - Rollback.
 - Threat model quando sensível.
 - Critério de acessibilidade quando visual.
 - Avaliação quando envolver agent.
 - Linhagem quando envolver dados.
 - Idempotência quando envolver workflow.
-

31. KPIs do programa

Plataforma

- Disponibilidade.
- Lead time de mudanças.
- Falhas de deploy.
- Tempo de recuperação.
- Workflow completion.

Dados

- Freshness.
- Completeness.
- Parse success.
- Lineage coverage.
- Quarentena.
- Reconciliação.
- Restatements tratados.

Agents

- Citation coverage.
- Claim support.
- Schema pass.
- Human override.
- Calibração.
- Custo por caso.
- Latência.
- Divergência.
- Incidentes de prompt injection.

Pesquisa

- Cobertura do universo.
- Teses ativas.
- Teses expiradas.
- Tempo até revisão.
- Evidências por tese.
- Conversão de oportunidades.

Carteiras

- Retorno líquido.
- Excesso.
- Drawdown.
- Risco.
- Turnover.
- Custos.
- Violações.
- Divergência paper/backtest.
- Divergência proposta/execução.

Decisões

- Tempo até comitê.
- Taxa de aprovação.
- Condições descumpridas.
- Decisões revertidas.
- Post-mortems.
- Valor incremental dos agents.

32. Ordem correta de construção

A sequência recomendada é:

1. Fazer o sistema iniciar
2. Corrigir banco, migrations e contratos
3. Tornar os dados confiáveis e point-in-time
4. Criar pesquisa, evidência e teses versionadas
5. Implementar agents limitados e avaliáveis
6. Criar domínio real de carteira e risco
7. Corrigir backtest e paper trading
8. Construir Mission Control e Top carteiras
9. Adicionar política e regulação
10. Consolidar comitê e governança
11. Operar paper
12. Considerar integração live

O frontend pode começar em paralelo com contratos mockados e Storybook, mas não deve transformar dados frágeis em dashboards aparentemente confiáveis.

A decisão arquitetural central deve permanecer:

Dados canônicos sustentam os fatos. Agents interpretam ambiguidades. Workflows controlam o processo. Policies limitam o que pode acontecer. O motor quantitativo calcula. O comitê decide. O painel explica e permite auditar tudo.